Swapr

SMART CONTRACT AUDIT REPORT

IMMUNEBYTES

21 June 2023

Introduction 3

About Lokr 3

About ImmuneBytes 3

Documentation Details 3

Audit Goals 4

Audit Process & Methodology 5

Audit Details 6

Security Level References 7

Finding 8

Critical Severity Issues 9

High Severity Issues 9

Medium severity issues 11

Low severity issues 13

Informational 17

Automated Audit 20

Concluding Remarks 22

Disclaimer 22

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Introduction

About Lokr

Lokr provides projects and individuals with a full suite of tools to manage their token economies. Whether it be token creation, distribution, creating cross-chain bridging solutions, or trustless escrows, Lokr has you covered. Lokr lowers the barrier of entry into the crypto space, making blockchain accessible to everyone through simple plug-and-play cross-chain solutions.

Visit https://www.lokr.io/ to know more about it.

About ImmuneBytes

ImmuneBytes is a security start-up that provides professional services in the blockchain space. The team has hands-on experience conducting smart contract audits, penetration testing, and security consulting. ImmuneBytes's security auditors have worked on various A-league projects and understand DeFi projects like AAVE, Compound, 0x Protocol, Uniswap, and dydx.

The team has secured 205+ blockchain projects by providing security services on different frameworks. The ImmuneBytes team helps start-ups with detailed system analysis, ensuring security and managing the overall project.

Visit http://immunebytes.com/ to learn more about the services.

Documentation Details

The team has provided the following doc for audit:

1. Notion Doc
2. https://github.com/Polkalokr/swapr-contracts/blob/audit/README.md

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Audit Goals

The focus of the audit was to verify that the smart contract system is secure, resilient, and working according to its specifications. The audit activities can be grouped into the following three categories:

1. Security: Identifying security-related issues within each contract and within the system of contracts.
2. Sound Architecture: Evaluation of the architecture of this system through the lens of established smart contract best practices and general software best practices.
3. Code Correctness and Quality: A full review of the contract source code. The primary areas of focus include

    a. Correctness
    b. Readability
    c. Sections of code with high complexity
    d. Quantity and quality of test coverage

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Audit Process & Methodology

ImmuneBytes team has performed thorough testing of the project, starting with analyzing the code design patterns in which we reviewed the smart contract architecture to ensure it is structured and safe use of third-party smart contracts and libraries.

Our team then performed a formal line-by-line inspection of the Smart Contract to find potential issues like Signature Replay Attacks, Unchecked External Calls, External Contract Referencing, Variable Shadowing, Race conditions, Transaction-ordering dependence, timestamp dependence, DoS attacks, and others.

In the Unit testing phase, we run unit tests written by the developer to verify the functions work as intended. In Automated Testing, we tested the Smart Contract with our in-house developed tools to identify vulnerabilities and security flaws.

The code was audited by a team of independent auditors which includes -

1. Testing the functionality of the Smart Contract to determine proper logic has been followed throughout.
2. Analyzing the complexity of the code by thorough, manual review of the code, line-by-line.
3. Deploying the code on testnet using multiple clients to run live tests.
4. Analyzing failure preparations to check how the Smart Contract performs in case of bugs and vulnerabilities.
5. Checking whether all the libraries used in the code are on the latest version.
6. Analyzing the security of the on-chain data.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Audit Details

Project Name	Lokr
Platform	EVM
Languages	Solidity
GitHub Link	https://github.com/Polkalokr/swapr-contracts
GitHub Branch	https://github.com/Polkalokr/swapr-contracts/tree/audit
Commit - Final Audit	29cd0056a677376f9f0413550073a002b9751ae4
Platforms & Tools	Remix IDE, Truffle, VScode, Contract Library, Slither, SmartCheck, Fuzz

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Security Level References

Every issue in this report was assigned a severity level from the following:

CRITICAL
Issues may result in fund leakage or incorrect fund allocation.

HIGH
Issues affecting the business logic, performance, and functionality.

MEDIUM
Issues could lead to data loss or other manipulations.

LOW
Issues around smart contract code upgradability, libraries, and others.

INFORMATIONAL
Issues which can further improve the code on gas optimizations and reusability.

OPEN
The issue hasn't been fixed as per our recommendation(s) and still stays pertinent.

FIXED
The issue has been agreed upon to be true and fixed as per our recommendation(s).

ACKNOWLEDGED
The issue is an intended behaviour and doesn't require a change as per our recommendation(s).

Issues	Critical	High	Medium	Low	Informational
Open	-	-	-	-	-
Closed	-	1	2	6	4
Acknowledged	-	-	-	-	-

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Finding

#	Findings	Risk	Status
1.	SwaprGL.createListing(): Absence of adequate validation checks before specific listings	High	Fixed
2.	SwaprGl.claim(): Function includes invalid timestamp check	Medium	Fixed
3.	SwaprGl.timeOffset(): Accurate value of timeOffset seems ambiguous	Medium	Fixed
4.	SwaprGL: Lack of Input Validation for Threshold Checks on uint Type State Variables	Low	Fixed
5.	SwaprFee.sol: Loosely Packed Struct found in the contract	Low	Fixed
6.	SwaprWallet.initialize() & SwaprGl.initialize: Absence of Zero Address validations	Low	Fixed
7.	SwaprGl, ListingHelper: Prefer Time Units and Ether Units	Low	Fixed
8.	SwaprGl: Tuple Mismatch in abi.decode().	Low	Fixed
9.	SwaprGl: Missing Error Messages in Require Statements	Low	Fixed
10.	ListingHelper._validateAsFirstBid(): Redundant logic can be avoided by using private function	Informational	Fixed
11.	SwaprFee.sol: Missing Event Emission after Modifying Crucial Arithmetic State Variables	Informational	Fixed
12.	Natspec Annotations should be accurate	Informational	Fixed
13.	Coding Style Issues in the Contract	Informational	Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Critical Severity Issues

No issues were found.

High Severity Issues

1. **SwaprGL.createListing(): Absence of adequate validation checks before specific listings**
   Line no: 100 to 105

Description:

The createListing() function in the contract allows the creation of new listings with specific parameters. For the listings to be accurate, the ListingHelper contract includes specific checkpoints that must be validated before creating new listings.

However, during the manual review, it was found that the listing validation functionalities do not adequately impose these validation checks in all scenarios.

The _validateListing() function of the ListingHelper contract will show that it mainly includes two specific validation functions, i.e., _validateAuctionCapability() and _validateOrderCapability().

A deep dive into both of these functions will indicate that there are two scenarios (2 conditional statements)that are considered in these functions:

a. First scenario: When NFTs are already locked in the SwaprWallet contract
b. Second Scenario: When they are freshly listed and aren't locked yet.

However, during the review, it was observed that not all parameters for an adequate listing are validated in the first scenario, i.e., in a case when NFTs are already locked.

For instance:

a. In the case of the _validateAuctionCapability() function, imperative parameters like seller's address, auction start time, or prices aren't validated in the first scenario.
b. Similarly, while the _validateOrderCapability() function doesn include some strict checks for both scenarios (line 416-422), but fails to validate all listing parameters for the first scenario.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

```
    Auction memory auction = _extractAuctionInfo(_listingData);

    if (swaprWallet.isNFTLocked(auction.lock, auction.nftId)) {
        require(
            auction.activeDepositType == 0 ||
                auction.activeDepositType == 3,
            "ALRD_LISTD"
        );
        isValid = true;
    } else {
        require(auction.depositType == 1, "INV_DEP_TYPE");
        require(auction.seller != address(0), "INV_SELLER");
        require(
            auction.startingPrice > 0 &&
                auction.buyNowPrice > auction.startingPrice,
            "INV_PRICE"
        );
        require(
            auction.startTime + timeOffset <= auction.endTime &&
                auction.endTime >= block.timestamp + timeOffset,
            "INV_TIME"
        );
        isValid = true;
    }
```

Inadequate validations of the listing parameters in the validation function of the ListingHelper contract will lead to quite a few unwanted scenarios in the marketplace and affect not just the createListing() but other functions like updatelisting() too.

Recommendation:

If the above-mentioned function design is not intentional, it is strictly recommended to provide adequate input validations for all listing parameters in all scenarios before the creation of any listing is executed.

While the function is guarded with signature verification, it still won't be considered a secure practice to remove strict validations for all listing scenarios in the contracts.

Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Medium severity issues

1. SwaprGl.claim(): Function includes invalid timestamp check **Line no: 178**
Description: The claim() function is specifically designed for buyers and sellers to claim their funds in an Auction-Type transaction. The function, at line 178, includes a strict checkpoint that ensures that the current timestamp must be greater than the end time of the bid. However, during the manual review, it was found that this require statement might be redundant as the _validateClaim function already includes a similar check (Line 248) to ensure that the current timestamp is greater than the listingEndTime stored in the bid. If this condition is not met, the function sets res to indicate an invalid claim and returns (false, res). Additionally, considering the current architecture of the contract it seems the actual require statement in the claim() function should have been to ensure the seller should be able to claim their funds after a particular auction's end time (auction.endTime) instead of the particular bid's end time (bid.listingEndTime), at line 178.
Recommendation: It is recommended to reevaluate the intended behavior of this function and design the function accordingly. If the above-mentioned suggestion aligns with the intended behavior of this function, then the require statement should be modified to impose the correct validations. Otherwise, redundant expressions should be omitted from functions.
Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

2. SwaprGl.timeOffset(): Accurate value of timeOffset seems ambiguous
Line no: 39

Description:

The SwaprGL contract includes an imperative state variable, i.e., timeOffset() to ensure accurate time tracking, which synchronizes the contract's internal time with an external time source if required.

However, it was found that the accurate value of this state variable seems ambiguous. While the code initializes this state variable with 5 mins, the comment indicates 15 minutes.

```
35        //All state variables must be initialized here in sequence to prevent upgrade conflicts
36        swaprWallet = ISwaprWallet(walletAddress);
37        _swaprFee = ISwaprFee(swaprFeeAddress);
38        theMarketplace = marketplaceAddress;
39        timeOffset = 5 * 60; //15 minutes // @audit - Indicates 15 mins but initializes to 5 mins
40        listingModTimeLimit = 60 * 60; //1 hour
41    }
42
```

Since the documentation doesn't clearly provide a specific value for the timeOffset state variable, it's imperative to take determine the correct value of this variable while deploying the contract as this plays a significant role in some crucial functions of the ListingHelper contract.

Recommendation:

Imperative variables should be initialized with accurate values. Contracts should follow proper natspec annotations standards to avoid readability issues.

Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Low severity issues

1. SwaprGL: Lack of Input Validation for Threshold Checks on uint Type State Variables
Description: The contract is missing input validation checks on the threshold values before updating uint-type state variables like timeOffset or listingModTimeLimit. This oversight can lead to potential vulnerabilities and unexpected behavior if the input values exceed certain thresholds or violate specific conditions. While these functions are marked with only owner access, it is still considered a best practice to include input validations to ensure the integrity and security of the contract.
Recommendation: To mitigate potential risks and improve the overall security of the contract, it is recommended to implement thorough input validation checks for threshold values before updating uint-type state variables. These checks should verify that the input values meet the required criteria and do not exceed predetermined limits or violate any specific conditions. Functions that lack such input validations: • updateTimeOffset() • setListingUpdateTime()
Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

2.	**SwaprFee.sol: Loosely Packed Struct found in the contract** **Line no: 20-29**
	Description: The FeePricing struct can be tightly packed to optimize storage usage. Currently, the struct members are not aligned in a way that minimizes storage slots. Each member is taking up a separate storage slot, resulting in unnecessary storage consumption.
	Recommendation: To optimize storage usage, it is recommended to tightly pack the FeePricing struct by aligning the struct members differently. By rearranging the members to minimize padding, the struct can be stored in a smaller number of storage slots.
	Status: Fixed

3.	**SwaprWallet.initialize() & SwaprGl.initialize: Absence of Zero Address validations**
	Description: The initialize function updates the swaprGLAddress state variable in the contract without any Zero Address Validation. Similarly, the imperative addresses in the SwaprGL contract are also initialized without proper validation. This can lead to potential security vulnerabilities or unexpected behavior in the contract.
	Recommendation: It is recommended to add explicit validation for zero addresses in the function arguments to ensure the contract behaves as intended and to prevent potential security issues. The validation can be done using a require statement or an if condition.
	Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

4. SwaprGl, ListingHelper: Prefer Time Units and Ether Units

Description:

The usage of explicit time units and ether units in smart contracts improves code readability and reduces the chance of errors.

It is recommended to use time units such as seconds, minutes, hours, or days instead of raw numbers when dealing with time-related calculations. Similarly, it is advisable to use ether units such as wei, ether, or finney instead of raw numbers when handling ether values.

Recommendation:

To enhance code clarity and consistency, replace the raw numbers with the corresponding time units or ether units in the contract.

Status: Fixed

5. SwaprGl: Tuple Mismatch in abi.decode(). Line no: 126

Description:

In the updateListing function, the type identifier specified for decoding the data input does not match the actual structure of the tuple returned by abi.decode.

While we are mainly expecting two bytes value, i.e., activeListingDate and proposedListingDate, the type identifier that specifies the expected structure of the tuple in the encoded data shows a 3 bytes value.

```
122         (
123             bytes memory activeListingData,
124             bytes memory proposedListingData,
125
126         ) = abi.decode(data, (bytes, bytes, bytes));
```

This mismatch doesn't really follow the best practices of solidity and might lead to incorrect decoding and unexpected behavior of the function.

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Recommendation:

To resolve this issue, ensure that the type identifier used in the abi.decode statement accurately reflects the structure of the tuple returned by abi.decode. Adjust the type identifier to match the expected number of elements and their respective types.

It is also essential and recommended to thoroughly test the function after making the necessary modifications to ensure proper decoding and functionality.

Status: Fixed

6. **SwaprGl: Missing Error Messages in Require Statements**
   Line no: 106, 150

Description:

The code contains require statements without any accompanying error messages. The require statements are used to validate conditions before proceeding with the execution of the code. However, without clear error messages, it becomes difficult to identify the specific reason for the requirement failure.

Recommendation:

It is recommended to include descriptive error messages in require statements. While missing error messages in require statements are not critical security vulnerabilities, they can affect user experience and make it harder to diagnose and fix potential issues.

Therefore, it is good practice to include descriptive error messages for better contract transparency and ease of debugging.

Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Informational

1. ListingHelper._validateAsFirstBid(): Redundant logic can be avoided by using private function

Description:

In the _validateAsFirstBid function, there is redundant logic to calculate the proposedEndTime.

This logic can be avoided by utilizing the existing private function _getProposedEndTime, which is already being used in the _bidsCrossValidate function for the same purpose.

```
    function _getProposedEndTime(
        uint listingEndTime
    ) private view returns (uint proposedEndTime) {
        if (listingEndTime - block.timestamp <= 60) {
            proposedEndTime = listingEndTime + timeOffset;
        } else {
            proposedEndTime = listingEndTime;
        }
    }
```

Recommendation:

To improve code readability and maintainability, it is recommended to remove the redundant logic and instead utilize the _getProposedEndTime function to calculate the proposedEndTime value in the _validateAsFirstBid function by passing the auction.endTime as a parameter to the private function.

Status: **Fixed**

2. SwaprFee.sol: Missing Event Emission after Modifying Crucial Arithmetic State Variables

Description:

Functions that update an imperative arithmetic state variable contract should emit an event after the state modification.
However, no events are emitted after modifying these crucial variables. This omission can make it challenging to track and monitor these state changes off-chain.

The following functions modify some crucial arithmetic parameters like in the contract but don't emit any event after that:

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

- setFixedBaseOrderFee
- setPercentBaseOrderFee
- setPercentBaseAuctionFee
- setFinalOrderFee
- setFinalAuctionFee

By emitting an event after modifying these variables, it becomes easier to capture and record the state changes off-chain.

Recommendation:

It is recommended to emit an event after modifying the crucial arithmetic state variables in the Masterchef contract. By doing so, the contract will provide a clear and auditable trail of state changes that can be tracked off-chain.

Status: Fixed

3. Natspec Annotations should be accurate

Description:

NatSpec annotations are used to provide clear and accurate explanations of the functionality and usage of smart contract functions.

However, during the code review, it was observed that the NatSpec annotations in some of the functions of the contract are not accurately reflecting the purpose and behavior of the functions.

For instance, SwaprFee.addPaymentToken() function allows two arguments while natspec annotations represent only one @param member.

Recommendation:

It is recommended to review and update the NatSpec annotations for all functions in the contract to ensure they accurately describe the purpose, behavior, and expected inputs/outputs of each function. The annotations should provide clear instructions, highlight potential risks, and offer guidance on proper usage.

Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

4. Coding Style Issues in the Contract
Description: The code style of a Smart Contract significantly impacts its readability and can have implications for future bug prevention. During the automated testing of the contract, it was observed that there are coding style issues present in the NAME contract. These issues include violations of naming conventions, inconsistent indentation, and improper code layout.
Recommendation: To enhance code readability and maintain consistent coding standards, it is recommended to address the coding style issues in the contract. This involves adhering to established naming conventions, ensuring proper indentation, and maintaining a consistent code layout throughout the contract. By following these best practices, the contract will become more understandable and maintainable, reducing the likelihood of potential bugs in the future. Please follow this link to find details on naming conventions in solidity code.
Status: Fixed

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

Automated Audit

1. SwaprGL

```
Compiled with solc
Number of lines: 3696 (+ 0 in dependencies, + 0 in tests)
Number of assembly lines: 0
Number of contracts: 28 (+ 0 in dependencies, + 0 tests)

Number of optimization issues: 0
Number of informational issues: 87
Number of low issues: 13
Number of medium issues: 12
Number of high issues: 3
ERCs: ERC165, ERC20, ERC721

+-----------------------------+-------------+---------------+--------------------+--------------+--------------------+
|             Name            | # functions |      ERCS     |     ERC20 info     | Complex code |      Features      |
+-----------------------------+-------------+---------------+--------------------+--------------+--------------------+
|      AddressUpgradeable     |      11     |               |                    |      No      |      Send ETH      |
|                             |             |               |                    |              |      Assembly      |
|       MathUpgradeable       |      14     |               |                    |     Yes      |      Assembly      |
|      StringsUpgradeable     |      5      |               |                    |      No      |      Assembly      |
|      IBeaconUpgradeable     |      1      |               |                    |      No      |                    |
|    StorageSlotUpgradeable   |      4      |               |                    |      No      |      Assembly      |
|  IERC20MetadataUpgradeable  |      9      |     ERC20     |     No Minting     |      No      |                    |
|                             |             |               | Approve Race Cond. |              |                    |
|                             |             |               |                    |              |                    |
|   IERC20PermitUpgradeable   |      3      |               |                    |      No      |                    |
|     SafeERC20Upgradeable    |      7      |               |                    |      No      |      Send ETH      |
|                             |             |               |                    |              | Tokens interaction |
|       ECDSAUpgradeable      |      10     |               |                    |      No      |     Ecrecover      |
|                             |             |               |                    |              |      Assembly      |
|     IERC1271Upgradeable     |      1      |               |                    |      No      |                    |
| SignatureCheckerUpgradeable |      1      |               |                    |      No      |                    |
|            ILock            |      14     | ERC165,ERC721 |                    |      No      |                    |
|          ISwaprFee          |      5      |               |                    |      No      |                    |
|         ISwaprWallet        |      15     |               |                    |      No      |    Receive ETH     |
|           SwaprGL           |      97     |     ERC165    |                    |      No      |    Receive ETH     |
|                             |             |               |                    |              |      Send ETH      |
|                             |             |               |                    |              |     Ecrecover      |
|                             |             |               |                    |              |    Delegatecall    |
|                             |             |               |                    |              | Tokens interaction |
|                             |             |               |                    |              |    Upgradeable     |
+-----------------------------+-------------+---------------+--------------------+--------------+--------------------+
```

2. SwaprFee

```
Compiled with solc
Number of lines: 2821 (+ 0 in dependencies, + 0 in tests)
Number of assembly lines: 0
Number of contracts: 22 (+ 0 in dependencies, + 0 tests)

Number of optimization issues: 0
Number of informational issues: 104
Number of low issues: 3
Number of medium issues: 22
Number of high issues: 2
ERCs: ERC165, ERC20

+----------------------------+-------------+--------+--------------------+--------------+--------------------+
|             Name           | # functions |  ERCS  |     ERC20 info     | Complex code |      Features      |
+----------------------------+-------------+--------+--------------------+--------------+--------------------+
|      AddressUpgradeable    |      11     |        |                    |      No      |      Send ETH      |
|                            |             |        |                    |              |      Assembly      |
|       MathUpgradeable      |      14     |        |                    |     Yes      |      Assembly      |
|      StringsUpgradeable    |      5      |        |                    |      No      |      Assembly      |
|      IBeaconUpgradeable    |      1      |        |                    |      No      |                    |
|    StorageSlotUpgradeable  |      4      |        |                    |      No      |      Assembly      |
| IERC20MetadataUpgradeable  |      9      | ERC20  |     No Minting     |      No      |                    |
|                            |             |        | Approve Race Cond. |              |                    |
|                            |             |        |                    |              |                    |
|  IERC20PermitUpgradeable   |      3      |        |                    |      No      |                    |
|    SafeERC20Upgradeable    |      7      |        |                    |      No      |      Send ETH      |
|                            |             |        |                    |              | Tokens interaction |
|   AggregatorV3Interface    |      5      |        |                    |      No      |                    |
|         IDIAOracleV2       |      1      |        |                    |      No      |                    |
|          SwaprFee          |     112     | ERC165 |                    |      No      |    Receive ETH     |
|                            |             |        |                    |              |      Send ETH      |
|                            |             |        |                    |              |    Delegatecall    |
|                            |             |        |                    |              | Tokens interaction |
|                            |             |        |                    |              |    Upgradeable     |
+----------------------------+-------------+--------+--------------------+--------------+--------------------+
```

This audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process, therefore running a bug bounty program as a complement to this audit is strongly recommended.

3. SwaprWallet

```
Compiled with solc
Number of lines: 2680 (+ 0 in dependencies, + 0 in tests)
Number of assembly lines: 0
Number of contracts: 25 (+ 0 in dependencies, + 0 tests)

Number of optimization issues: 0
Number of informational issues: 84
Number of low issues: 4
Number of medium issues: 11
Number of high issues: 4
ERCs: ERC721, ERC165, ERC20

+---------------------------+-------------+---------------+--------------------+--------------+--------------------+
|            Name           | # functions |      ERCs     |     ERC20 info     | Complex code |      Features      |
+---------------------------+-------------+---------------+--------------------+--------------+--------------------+
|     AddressUpgradeable    |      11     |               |                    |      No      |      Send ETH      |
|                           |             |               |                    |              |      Assembly      |
|      MathUpgradeable      |      14     |               |                    |     Yes      |      Assembly      |
|     StringsUpgradeable    |      5      |               |                    |      No      |      Assembly      |
|     IBeaconUpgradeable    |      1      |               |                    |      No      |                    |
|   StorageSlotUpgradeable  |      4      |               |                    |      No      |      Assembly      |
| IERC20MetadataUpgradeable |      9      |     ERC20     |     No Minting     |      No      |                    |
|                           |             |               | Approve Race Cond. |              |                    |
|                           |             |               |                    |              |                    |
|  IERC20PermitUpgradeable  |      3      |               |                    |      No      |                    |
|    SafeERC20Upgradeable   |      7      |               |                    |      No      |      Send ETH      |
|                           |             |               |                    |              | Tokens interaction |
|           ILock           |      14     | ERC165,ERC721 |                    |      No      |                    |
|       ISplitManager       |      1      |               |                    |      No      |                    |
|        SwaprWallet        |      86     |     ERC165    |                    |      No      |    Receive ETH     |
|                           |             |               |                    |              |      Send ETH      |
|                           |             |               |                    |              |    Delegatecall    |
|                           |             |               |                    |              | Tokens interaction |
|                           |             |               |                    |              |    Upgradeable     |
+---------------------------+-------------+---------------+--------------------+--------------+--------------------+
```

4. ListingHelper

```
Compiled with solc
Number of lines: 1718 (+ 0 in dependencies, + 0 in tests)
Number of assembly lines: 0
Number of contracts: 11 (+ 0 in dependencies, + 0 tests)

Number of optimization issues: 2
Number of informational issues: 44
Number of low issues: 5
Number of medium issues: 8
Number of high issues: 3
ERCs: ERC721, ERC165

+----------------------------+-------------+---------------+------------+--------------+--------------------+
|            Name            | # functions |      ERCs     | ERC20 info | Complex code |      Features      |
+----------------------------+-------------+---------------+------------+--------------+--------------------+
|       MathUpgradeable      |      14     |               |            |     Yes      |      Assembly      |
|                            |             |               |            |              |    Upgradeable     |
|      StringsUpgradeable    |      5      |               |            |      No      |      Assembly      |
|                            |             |               |            |              |    Upgradeable     |
|       ECDSAUpgradeable     |      10     |               |            |      No      |     Ecrecover      |
|                            |             |               |            |              |      Assembly      |
|                            |             |               |            |              |    Upgradeable     |
|     IERC1271Upgradeable    |      1      |               |            |      No      |    Upgradeable     |
|      AddressUpgradeable    |      11     |               |            |      No      |      Send ETH      |
|                            |             |               |            |              |      Assembly      |
|                            |             |               |            |              |    Upgradeable     |
| SignatureCheckerUpgradeable |      1      |               |            |      No      |    Upgradeable     |
|           ILock            |      14     | ERC165,ERC721 |            |      No      |    Upgradeable     |
|         ISwaprWallet       |      15     |               |            |      No      |    Receive ETH     |
|        ListingHelper       |      20     |               |            |      No      |     Ecrecover      |
|                            |             |               |            |              | Tokens interaction |
+----------------------------+-------------+---------------+------------+--------------+--------------------+
```

Concluding Remarks

While conducting the audits of the Lokr - Swapr, it was observed that the contracts contain High, Medium, and Low severity issues along with a few recommendations.

Our auditors suggest that the developers should resolve High, Medium, and Low severity issues. The recommendations given will improve the operations of the smart contract.

Note: The Lokr team has fixed the issues based on the auditor's recommendations.

Disclaimer

ImmuneBytes' audit does not provide a security or correctness guarantee of the audited smart contract. Securing smart contracts is a multistep process; therefore, running a bug bounty program complementing this audit is strongly recommended.

Our team does not endorse the Lokr platform or its product, nor is this audit investment advice.
Notes:

- Please make sure contracts deployed on the mainnet are the ones audited.
- Check for code refactoring by the team on critical issues.

IMMUNEBYTES

AUDITS